

SRE Capstone Project

Production Readiness Review

`ecom-api` — a Go e-commerce microservice

Group SE2408

Adildabek Nurassyl · Artem Safaryan

1 · Objective

Take a newly developed microservice through a real **Production Readiness Review** by demonstrating all four SRE pillars on a reproducible local stack:

-  **Infrastructure as Code** — Terraform
-  **CI/CD** — GitHub Actions → `ghcr.io`
-  **Observability** — Prometheus, Grafana, Alertmanager
-  **SRE Operations** — SLI/SLO, auto-scaling, load testing

Everything runs on a laptop in ~5 minutes. No cloud account required.

2 · Architecture

```
Terraform → Kind cluster (3 nodes)
            └─ ns: ecom      (ecom-api · HPA · ServiceMonitor · PrometheusRule)
            └─ ns: monitoring (kube-prometheus-stack: Prom + Grafana + Alertmanager)

GitHub Actions → ghcr.io/RendersC/ecom-api:<sha> → pods
Locust         → http://localhost:30081 (NodePort)
Browser        → Grafana :30030 · Prometheus :30090 · Alertmanager :30080
```

- Kind v1.30 · Go 1.22 · kube-prometheus-stack 61.x · HPA v2 + metrics-server

3 · Step 1 — Infrastructure as Code

Provisioned by Terraform:

Resource	Purpose
<code>kind_cluster.this</code>	1 control-plane + 2 workers
<code>helm_release.metrics_server</code>	Enables HPA
<code>helm_release.kube_prometheus_stack</code>	Prom + Grafana + Alertmanager + Operator
<code>kubernetes_namespace.{app, monitoring}</code>	Namespaces with labels

- **Reproducible** — `terraform apply` builds the full stack from zero in ~4 min
- **Pinned** — provider, chart, and image versions all in `variables.tf`

4 · Step 2 — CI/CD Pipeline

```
push to main → test → build & push → deploy
                |           |           |
                |           |           |
            go vet      Buildx →      patch k8s/02-deployment.yaml
            go test      ghcr.io:<sha>  with new tag, upload artifact
                        + :latest
```

- **No external secrets** — uses GitHub's `GITHUB_TOKEN` for `ghcr.io`
- **Immutable tags** — every push produces `ghcr.io/RendersC/ecom-api:<sha>`
- **Distroless image** — ~12 MB, non-root, read-only root FS

5 · Step 3 — Observability (1/2)

Metrics instrumented in the Go app (`prometheus/client_golang`):

Metric	Type	Purpose
<code>http_requests_total{route,method,status}</code>	counter	RED — Requests + Errors
<code>http_request_duration_seconds</code>	histogram	RED — Duration
<code>http_inflight_requests</code>	gauge	Saturation
<code>orders_created_total</code> / <code>orders_failed_total</code>	counters	Business SLI

Recording rules (`PrometheusRule`):

`ecom_api:request_rate:5m` · `ecom_api:error_rate:5m` · `ecom_api:latency_p95:5m`

6 · Step 3 — Observability (2/2)

Grafana — SRE Golden Signals dashboard:

- Error rate, p95 latency, request rate, Ready pods (single-stat)
- Request rate by route · Latency p50/p95/p99 by route
- Error ratio time series · Pod count vs HPA desired

Alerts (Alertmanager):

Alert	Threshold	Severity
EcomApiHighErrorRate	5xx ratio > 5% for 2m	critical
EcomApiHighLatencyP95	p95 > 500ms for 5m	warning
EcomApiPodNotReady	0 Ready pods for 2m	critical
EcomApiHighCpu	per-pod CPU > 200m for 5m	warning

7 · Step 4 — SLI / SLO

SLOs (28-day rolling window):

SLI	Target	Error budget
Availability (non-5xx)	99.5%	≈ 3h 21m / 28d
Latency p95 — /api/products*	< 300 ms	5% of 5-min windows
Latency p95 — /api/orders	< 500 ms	5% of 5-min windows
Order success rate	≥ 99%	1% of orders

Error-budget policy: ship freely → risk review → code freeze → incident.

8 · Step 4 — Auto-scaling & Load test

HPA v2 — `minReplicas: 2 · maxReplicas: 8`

- CPU target 60% · Memory target 75%
- scaleUp: +2 pods / 30s · scaleDown: -1 pod / 60s with 180s stabilization

Locust — staged spike profile demonstrates HPA in action:

```
0-60s    → 30 users    (warm-up)
60-180s  → 80 users    (steady)
180-300s → 250 users   (SPIKE — HPA triggers)
300-420s → 80 users    (cool down)
```

During the spike: pods go 2 → 6 in ~90s, p95 stays under SLO.

9 · Live Demo Plan (~8 min)

Min	Action
0:00	<code>bootstrap</code> output / <code>kubectl get pods -A</code>
1:00	<code>smoke-test.sh</code> — API + metrics
1:30	Grafana SRE Golden Signals walk-through
2:30	Prometheus Targets + Alerts
3:00	Alertmanager UI
3:30	Start Locust staged spike
5:00	Watch HPA scale 2 → 6 pods live
6:30	Show an alert firing
7:30	GitHub Actions green run + <code>ghcr.io</code> package

10 · Team contributions

Adildabek Nurassyl — IaC & Observability

- Terraform stack (Kind, metrics-server, kube-prometheus-stack)
- ServiceMonitor, PrometheusRule, recording rules
- Grafana dashboard, Alertmanager routing
- Architecture, SLO, runbook docs

Artem Safaryan — Application & CI/CD

- Go microservice with Prometheus instrumentation
- Distroless Dockerfile, K8s Deployment/Service/HPA
- GitHub Actions pipeline → `ghcr.io`
- Locust load test + smoke test

11 · Known limitations / next steps

Out of scope	Next step
Persistent DB	Add Postgres via Bitnami chart + PVC
Multi-AZ / cloud	Swap Kind for EKS Terraform module
Real alert channels	Replace placeholder webhooks with Slack/PagerDuty
TLS / Ingress	Add cert-manager + ingress-nginx
Multi-window burn-rate alerts	Add Google SRE workbook recipes

Thank you · Questions?

Repository: `github.com/RendersC/sre-final`

Live demo: Grafana `:30030` · Prometheus `:30090` · Alertmanager `:30080`

Adildabek Nurassyl · Artem Safaryan

Group SE2408